VITYARTHI — Open Source Software Course | 2025–26

The Open Source Audit

A Capstone Project for OSS NGMC Course

Student Name	Yash Tripathi
Registration Number	24BCE10603
Slot	B22
Date of Submission	29 March 2026
Chosen Software	Git — Version Control System
Licence	*GNU GPL v2*

Part A — Origin and Philosophy

A1. The Problem Git Was Created to Solve

How It Actually Started

What people usually say is that Linus just needed a version control system, but that version skips over most of what actually made the situation interesting.

Back in 2002, the Linux kernel team started using a proprietary tool called BitKeeper. It was developed by a company named BitMover, and they allowed the community to use it for free. But it wasn't "free" in the real sense — it was more like they were allowing access on their own terms. People knew this wasn't ideal, but since the tool worked well and there weren't many good alternatives, everyone just went along with it.

Things stayed like that for a few years. Then in April 2005, Andrew Tridgell tried to reverse-engineer BitKeeper's protocol to create an open alternative. BitMover didn't take that lightly and quickly revoked access for the entire Linux kernel project. Suddenly, thousands of developers were left without a version control system.

At that point, Linus Torvalds didn't have much time — the next kernel merge window was coming up fast. He checked the existing tools, but none of them really fit. CVS was something he openly disliked, and although Subversion was better, it still wasn't good enough for what they needed. Performance and the lack of a distributed model were major issues.

So instead of settling, he decided to build something himself. In about ten days, the first version of Git was ready. Within weeks, the Linux kernel project had already switched over to it.

Why 'Distributed' Was the Point

Before Git came along, most version control systems like CVS, Subversion, and Perforce followed a client-server approach. Basically, everything depended on a single central server. If you wanted to commit changes, check history, or even create branches, you had to connect to that server. This created some obvious problems — if the server went down, work would stop completely. If it was slow, everyone felt it. And more importantly, the person or organisation controlling that server effectively controlled the entire project.

Git changed this idea in a big way. Instead of relying on one central place, every developer gets a complete copy of the repository, including its full history. So even if you're offline, you can keep working, make multiple commits, and only sync later when you're connected again. There isn't a single server that everything depends on anymore.

This wasn't just about making things faster or more efficient. It was also a reaction to what happened with BitKeeper. The Linux project had depended on a proprietary system, and when access was taken away, everything came to a halt. Git was designed to avoid that kind of situation entirely — no single point where things can break, and no single authority controlling everything. That idea is deeply built into how Git works.

What Git Actually Changed

I feel like people often underestimate how different things were before Git. In tools like Subversion, branching wasn't something you could just do casually — it took time and added complexity, so teams usually avoided it. Most of the work happened on a single main branch, and developers had to coordinate carefully to avoid conflicts. Merging changes was often messy and could easily go wrong. The idea of having lots of parallel branches, with people working independently and merging whenever they wanted, just wasn't really practical back then.

Git changed that completely. Creating a branch takes almost no time, and merging is much smoother. Because of that, the way people collaborate — especially in open source — shifted a lot. GitHub, which came later, is basically built around this idea of easy branching. Even the pull request workflow that most projects use today depends on the fact that branching is so lightweight.

Sometimes when I run git log and see a long history of contributions from so many different people, all organised and easy to track, it's kind of surprising. It all traces back to the design decisions Torvalds made in a very short time back in 2005. It's a bit strange to think that something built so quickly ended up shaping how almost all software is developed today.

A2. The Licence — What It Actually Says

The Four Freedoms — and Whether Git Actually Delivers Them

The Free Software Foundation defines four freedoms that a piece of software must grant for it to be considered 'free software':

- Freedom 0 — the freedom to run the program for any purpose
- Freedom 1 — the freedom to study the source code and change it
- Freedom 2 — the freedom to redistribute copies
- Freedom 3 — the freedom to distribute your modified versions

Git is licensed under the GNU General Public Licence version 2 (GPL v2), and it grants all four. You can download Git's entire source code from kernel.org, read it, modify it, and ship your own version. There are no restrictions on what you use it for — a Fortune 500 company and a student in a hostel room have exactly the same rights over Git. That's what these freedoms mean in practice.

Can a Company Take Git, Modify It, and Sell It Without Sharing?

No. And I think this is the most important thing to understand about the GPL, because it's where it differs sharply from licences like MIT or Apache.

The GPL uses something called a copyleft clause. In simple terms, if you modify Git and then distribute that version to others, you're required to release your source code under the same GPL v2 licence. So you can't just take Git, improve it, and then turn it into a closed, proprietary product. The idea is that the freedom doesn't stop with the first user — it continues down the line to everyone who receives the software.

Companies like Red Hat show how this can still work commercially. They build products using GPL software, but instead of making money by restricting access to the code, they focus on support, services, and expertise. Their business model depends on the code staying open, not hidden. There have also been cases where companies tried to bypass these rules and were forced to release their code after legal action from organisations like the Free Software Foundation.

Personally, I think this is what gives the GPL its real strength. Licences like MIT and Apache are more flexible for businesses since they allow code to be taken and made proprietary. That's not necessarily wrong, but it does mean community work can end up being closed off later. The GPL avoids that by enforcing a simple idea: if you benefit from the freedom, you have to pass it on as well.

GPL vs MIT — Which Would I Actually Choose?

This is something I actually spent some time thinking about. If I were building a library meant for other developers to use in their own projects, I'd probably go with the MIT licence. The main reason is

practicality — if you want people to adopt what you've built, a permissive licence makes that easier. Companies are usually cautious with GPL-licensed code because of the legal obligations, whereas MIT removes most of that friction.

On the other hand, if I were working on something I considered important infrastructure — something I wouldn't want a company to take, modify, and close off later — then I'd lean towards GPL. Projects like Git and the Linux kernel fall into that category. The people behind them clearly wanted to make sure the code would always remain open, no matter who used it or built on top of it. I can see why they made that choice, and it makes sense in that context.

At the end of the day, there isn't a single "correct" licence that works for everything. It really depends on your priorities. MIT is great if your focus is wide adoption, while GPL is more about preserving openness and ensuring those freedoms continue.

Free as in Free Beer vs Free as in Freedom

This distinction can sound a bit theoretical at first, but it becomes very real in certain situations.

When people say "free as in free beer," they just mean something that doesn't cost money. But "free as in freedom" — often called *libre* — is about what you're actually allowed to do with the software. A company could offer a version control tool for free, and on the surface that sounds great. But if you can't see the source code, can't modify how it works, or can't leave without losing access to your own data, then you're not really in control. That's not the kind of freedom that open source talks about.

Git happens to be both free in cost and free in terms of freedom, but the second part is what really matters. A good example is what happened in 2005 with the Linux project. They were using BitKeeper, which was free to use at the time, but still proprietary. As soon as the company behind it changed its decision, access was gone. That left the entire project stuck overnight.

Git was created to avoid exactly that kind of situation. With true freedom, no single company can suddenly take the tool away from you. And that difference — between just being free to use and actually having control — ends up mattering a lot more than it might seem at first.

A3. The Ethics of Open Source

Should All Software Be Open Source? My Actual Position

I'd rather take a clear position on this than give a safe, middle-of-the-road answer.

I don't think every piece of software needs to be open source. But at the same time, I do believe that software which acts as critical infrastructure should be. Tools like Git, Linux, and OpenSSL are not just ordinary applications — a huge part of the internet depends on them. The idea that something so fundamental could be controlled by a single private company is a bit concerning. In cases like this, open source isn't just a preference, it's more about public interest. The Heartbleed bug in OpenSSL is a good example — because the code was open, people could quickly understand the issue and work on fixing it. That level of transparency really matters.

For other types of software, like games or specialised creative tools, the situation is different. Those often rely on commercial incentives, and making everything open source might not be practical. I use proprietary software myself, so I'm not arguing that everything should be open by default. But there's a

clear difference between general consumer software and infrastructure-level software, and I think it makes sense to expect a higher level of openness from the latter.

Is It Ethical to Build a Profitable Company on Community Labour?

This is the uncomfortable question, and I don't think the easy answer — 'the licence allows it, so it's fine' — is enough.

A lot of large tech companies rely heavily on open-source software, and in many cases, they've gained huge benefits from it without always giving back in equal measure. Take Amazon Web Services as an example — much of its infrastructure is built on tools like Linux, Apache, and Python, which are maintained by communities. For a long time, it felt like they were benefiting more than they were contributing. To be fair, they've increased their contributions over time, but that shift didn't really happen until there was noticeable pressure from the community.

I think there's an important difference between companies that both use and contribute, and those that mostly just use. Companies like Red Hat actively invest in open source by hiring developers and contributing code back to projects. Others may follow the rules of the licence but don't contribute much in return. Technically, both approaches are allowed, but they're not the same. Open source works a bit like a shared resource — it needs ongoing effort to stay healthy — and not everyone helps maintain it.

Personally, I feel that companies making significant profits from open-source software should contribute back in some way. It may not be a legal requirement, but there's a strong ethical argument for it. The people maintaining projects like Git aren't doing it so that large corporations can use it for free at scale — they're doing it because they care about the work. Given that, it seems reasonable that the biggest beneficiaries should help support and sustain those projects.

Standing on the Shoulders of Giants

Pretty much every piece of software today builds on something that came before it. Git is a good example of that — it wasn't created completely from scratch. Torvalds took ideas from systems like Monotone and BitKeeper, used existing concepts like SHA-1 hashing, and followed the Unix idea of keeping tools simple and focused. What he really did was bring all of that together in a way that worked better.

Open source makes this kind of progress much easier. When the code is available, developers can actually look at how things were done, understand the reasoning behind certain decisions, and improve on them instead of starting from zero. With proprietary software, that visibility isn't there, so different teams often end up solving the same problems again and again, but in isolation.

To me, this is why open source tends to move things forward more efficiently overall. Individual companies might move faster in specific areas by keeping things closed, but at a broader level, shared knowledge helps everyone improve faster. That's really what "standing on the shoulders of giants" means in practice — not just using past work, but being able to learn from it and build on it in a meaningful way.

Part B — Linux Footprint

Installation

Git is available through all major Linux package managers. On a Debian/Ubuntu system the command is 'sudo apt install git'. On Fedora or RHEL it's 'sudo dnf install git'. On Arch it's 'sudo pacman -S git'. For those who want the very latest version or need custom compile options, building from source is also possible — download the tarball from git-scm.com and run ./configure, make, then sudo make install.

The package-manager route is what I'd recommend for most cases. It ties Git into the system's update mechanism, which means security patches land automatically alongside the rest of your system updates. That's not a small thing — Git has had security vulnerabilities, and prompt patching matters.

Key Directories

After installation, Git distributes itself across several directories following the Linux Filesystem Hierarchy Standard:

Directory	Contents	What it's for
/usr/bin/git	Main binary	The git command you actually run
/usr/lib/git-core/	Sub-commands	git-pull, git-push, git-fetch etc.
/usr/share/git-core/	Templates	Default hooks and template files
/etc/gitconfig	System config	Settings that apply to every user
~/.gitconfig	User config	Your personal name, email, aliases
.git/ (per repo)	Repository data	All commits, history, branches, refs

The .git directory inside each repository is particularly worth understanding. Everything Git knows about a project lives there — the complete history, all branches, all tags, the staging area. If you delete it, the project becomes a plain folder with no version history. If you copy it somewhere else and run 'git checkout', you can reconstruct any version of the project that was ever committed. That's how Git's distributed model works.

User, Group, and Security

For local usage, Git runs as whatever user is currently logged in. There's no dedicated system user for Git in that context. On a server hosting shared repositories, however, a dedicated 'git' user account is typically created, and that user is given a restricted shell called git-shell. This means that even if someone gains SSH access as the git user, they can only interact with repositories — they cannot run arbitrary commands on the server.

Repository directories on a shared server are owned by git:git, usually with permissions 755 or sometimes 750 depending on whether repositories should be readable by other local users. This follows the principle of least privilege — the git account has exactly the access it needs and nothing more.

Service Management

Git is not a daemon — it doesn't run continuously in the background during normal usage. But when used as a server, it's commonly paired with services that do. The three main options:

- git daemon — a lightweight read-only server for the Git protocol. Managed with: sudo systemctl start git-daemon.socket
- SSH — the most common way to host Git remotely. You're actually managing sshd, not Git directly: sudo systemctl status sshd

- Gitea or GitLab — full web-based hosting platforms, both managed as systemd services: sudo systemctl start gitea

Update Model

Security updates for Git usually come through the normal Linux update system. When a vulnerability is discovered and fixed by the Git developers, a new version gets released. After that, distribution maintainers package the update and push it to their repositories. From a user's side, it just shows up through commands like apt upgrade or dnf upgrade, and in many setups, this happens automatically.

One thing that stands out here is how transparent the whole process is. The vulnerability is made public, the fix is visible, and anyone can trace what happened and how it was resolved. It's quite different from proprietary software, where you're mostly expected to trust whatever update is provided. With open source, people can actually look at the changes and verify them themselves, which makes the process a lot more open and trustworthy.

Part C — The FOSS Ecosystem

What Git Depends On

For something so widely used, Git's dependencies are surprisingly lean. This is partly why it runs on everything from embedded systems to mainframes.

- OpenSSL or GnuTLS — handles the cryptographic side of HTTPS connections when you push or pull over HTTPS.
- libcurl — manages the actual HTTP/HTTPS communication for Git's smart HTTP protocol.
- zlib — compresses objects before storing or transmitting them. This is why Git repositories are compact relative to the actual file sizes they track.
- glibc — the standard C library. Git's core is written in C and relies on it for everything from file I/O to memory management.
- Perl — some of the older Git scripts and the git-svn bridge are written in Perl.

What Git Has Made Possible

If I had to answer honestly, I'd say a huge part of modern software development depends on Git in some way. GitHub, which started in 2008, now hosts hundreds of millions of repositories. GitLab came along as an alternative, and the fact that you can self-host it is a big deal for organisations that can't rely on external servers. Both platforms are built entirely around Git, so without it, they wouldn't really exist in the same form.

Tools like GitHub Actions and GitLab CI also show how much Git has shaped workflows. These systems automatically run tests or deploy code whenever someone pushes a commit. That kind of automation has become normal now, but it wasn't always like this. Continuous integration existed before, but Git made it feel like a default part of development rather than something extra.

More recently, platforms like Hugging Face have started using Git as well, especially with Git LFS for handling large files like machine learning models. Git wasn't originally designed with machine learning in mind, but it turns out the core idea was flexible enough to support it anyway. It's interesting how something built for one purpose ended up being useful in areas that didn't even exist at the time.

Git in the Web Stack

Git doesn't appear in the LAMP acronym — it's not a runtime component alongside Linux, Apache, MySQL, and PHP. But it's present at every stage of how web applications are built and deployed. The PHP code running on Apache was written in a Git repository. The deployment pipeline that pushed it to the server ran a 'git pull' or 'git clone'. The configuration files are version-controlled in Git. The infrastructure-as-code scripts that provisioned the server are in Git.

It's invisible in the stack diagram and present in virtually every step of building one.

Community and Governance

Git's development happens primarily on a public mailing list — git@vger.kernel.org — where patches are submitted, reviewed, and discussed. Junio C. Hamano has been the primary maintainer since Torvalds handed stewardship over in 2005. He's not employed by any single company — he works at Google, but maintains Git independently of that.

This governance model is worth noting. There's no corporation that owns Git. No board that could vote to close it. No acquisition that could change its licence. The project is governed by the maintainers and community, and the GPL licence ensures that even if everyone involved today disappeared, anyone could pick up the code and continue.

Microsoft's 2018 acquisition of GitHub raised eyebrows, but Git itself was unaffected — Microsoft doesn't own Git, they own a web platform built on top of it. They have since become significant contributors to the Git project, particularly improving Windows compatibility and performance for very large repositories.

Part D — Open Source vs Proprietary

Git vs Perforce Helix Core

Perforce is the most widely used proprietary version control system, particularly in game development and large enterprises. It's worth comparing honestly because 'open source is always better' is not actually a defensible position — there are real trade-offs.

Dimension	Git (Open Source)	Perforce Helix Core (Proprietary)
Cost	Free — zero licence fees, forever	Per-user licensing; expensive at scale
Code auditability	Full source available at kernel.org	Closed source — trust the vendor
Support	Community + paid support via GitHub, GitLab etc.	Vendor SLA; dedicated enterprise support team
Large binary files	Needs Git LFS extension; manageable but not seamless	Native and strong — built for game/media workflows
Freedom to modify	Complete — fork, patch, redistribute under GPL	None — source is not available

Ecosystem	Enormous — GitHub, GitLab, every CI/CD tool, every IDE	Smaller; mainly enterprise game studios
Community control	No single owner; no vendor lock-in possible	Corporate roadmap; licence can change at any time
Overall verdict	Right choice for virtually all software development	Justified only for very large game/media asset pipelines

My Verdict

If I'm being honest, for most software development, it's hard to argue against Git. The ecosystem around it is just too strong — almost every tool integrates with Git, and pretty much every developer already knows how to use it. The whole collaboration model we see on platforms like GitHub and GitLab, especially with branching and pull requests, really depends on how Git works. There isn't a direct equivalent to that in the Perforce world.

That said, Perforce does have a clear advantage in one area — handling large binary files. In industries like AAA game development, where teams deal with massive assets like textures, 3D models, and audio files, Perforce tends to work better. Git wasn't really designed for that kind of workload. Git LFS helps, but it still doesn't feel as seamless as Perforce's built-in approach. So in that specific scenario, I can see why a team might choose Perforce.

For most other use cases though, I'd still go with Git without much hesitation. And contributing to it also feels worthwhile — not just as an academic requirement, but because improvements to a project like Git end up helping a huge number of developers. That's really the core idea behind open source, and it's a big part of why it's so effective.

Part E — Shell Script Documentation

Script 1 — System Identity Report

Explanation:

This script displays a welcome screen with information about the Linux system — distribution name, kernel version, current user, home directory, uptime, date and time, and a message about the GPL v2 licence covering both Linux and Git.

Concepts demonstrated

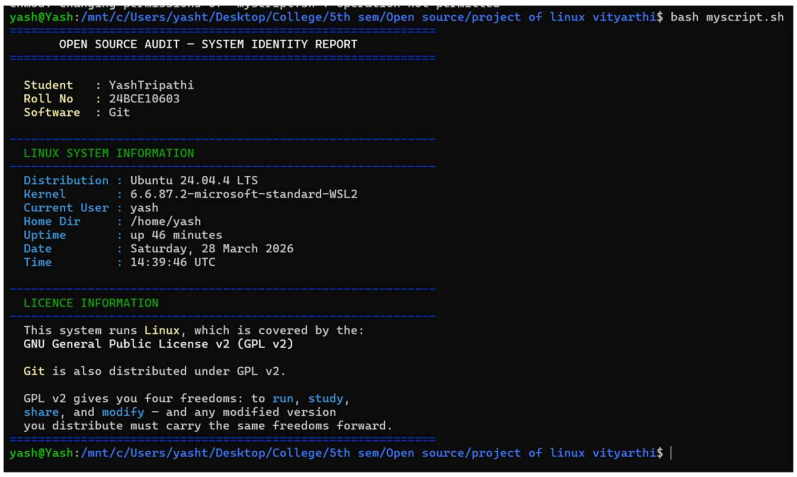
- Variables — STUDENT_NAME, ROLL_NUMBER, SOFTWARE_CHOICE store values used across the script.
- Command substitution — $(uname -r), $(whoami), $(uptime -p), $(date) run commands and capture their output into variables at runtime.
- Output formatting — echo statements with consistent spacing and separator lines produce readable display.
- cat /etc/os-release with grep and cut extracts the human-readable distribution name.

Code:

```
#!/bin/bash
# ==============================================================
# Script 1: System Identity Report
# Course  : Open Source Software — VITyarthi
# Author  : Yash Tripathi | Roll No: 24BCE10603
# What it does: Prints a welcome screen showing the Linux
#               system identity and the licence covering it.
```

```
# ==============================================================

# ANSI colour codes — makes output readable at a glance
GREEN='\033[0;32m'
BLUE='\033[0;34m'
YELLOW='\033[1;33m'
CYAN='\033[0;36m'
BOLD='\033[1m'
NC='\033[0m'    # No Colour — always reset after each coloured line
# Student details — fill these in
STUDENT_NAME=YashTripathi
ROLL_NUMBER=24BCE10603
SOFTWARE_CHOICE="Git"
SOFTWARE_LICENSE="GNU General Public License v2 (GPL v2)"
# Gather live system info using command substitution — $(command)
# runs the command and places its output directly into the variable
KERNEL=$(uname -r)
DISTRO=$(grep -w "PRETTY_NAME" /etc/os-release 2>/dev/null | cut -d= -f2 | tr -d '"')
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
CURRENT_DATE=$(date '+%A, %d %B %Y')
CURRENT_TIME=$(date '+%H:%M:%S %Z')
# Print the header banner
echo -e "${BLUE}==============================================================${NC}"
echo -e "${BOLD}       OPEN SOURCE AUDIT — SYSTEM IDENTITY REPORT          ${NC}"
echo -e "${BLUE}==============================================================${NC}"
echo ""
echo -e "  ${YELLOW}Student   :${NC} $STUDENT_NAME"
echo -e "  ${YELLOW}Roll No   :${NC} $ROLL_NUMBER"
echo -e "  ${YELLOW}Software  :${NC} $SOFTWARE_CHOICE"
echo ""
echo -e "${BLUE}--------------------------------------------------------------${NC}"
# Linux system section
echo -e "  ${GREEN}LINUX SYSTEM INFORMATION${NC}"
echo -e "${BLUE}--------------------------------------------------------------${NC}"
echo -e "  ${CYAN}Distribution :${NC} $DISTRO"
echo -e "  ${CYAN}Kernel       :${NC} $KERNEL"
echo -e "  ${CYAN}Current User :${NC} $USER_NAME"
echo -e "  ${CYAN}Home Dir     :${NC} $HOME_DIR"
echo -e "  ${CYAN}Uptime       :${NC} $UPTIME"
echo -e "  ${CYAN}Date         :${NC} $CURRENT_DATE"
echo -e "  ${CYAN}Time         :${NC} $CURRENT_TIME"
echo ""
echo -e "${BLUE}--------------------------------------------------------------${NC}"
# Licence section
echo -e "  ${GREEN}LICENCE INFORMATION${NC}"
echo -e "${BLUE}--------------------------------------------------------------${NC}"
```

```
echo -e "  This system runs ${YELLOW}Linux${NC}, which is covered by the:"
echo -e "  ${BOLD}$SOFTWARE_LICENSE${NC}"
echo ""
echo -e "  ${YELLOW}$SOFTWARE_CHOICE${NC} is also distributed under GPL v2."
echo ""
echo -e "  GPL v2 gives you four freedoms: to ${CYAN}run${NC}, ${CYAN}study${NC},"
echo -e "  ${CYAN}share${NC}, and ${CYAN}modify${NC} — and any modified version"
echo -e "  you distribute must carry the same freedoms forward."
echo -e "${BLUE}==============================================================${NC}"
```



Script 2 — FOSS Package Inspector

Explanation:

This script checks whether Git is installed, handles both RPM-based and Debian-based package managers, prints version and licence metadata, and uses a case statement to display a philosophy note about whichever open-source package is named.

Concepts demonstrated

- if-then-else — tests whether rpm or dpkg is available using command -v, then queries the appropriate one.
- case statement — matches the package name against known open-source tools and prints a relevant note for each.
- Pipe with grep — rpm -qi $PACKAGE | grep -E filters package metadata to show only the relevant fields.
- &>/dev/null — suppresses both stdout and stderr when silently testing whether a command succeeds.

Code:

```
#!/bin/bash
# ==============================================================
# Script 2: FOSS Package Inspector
# Course  : Open Source Software — VITyarthi
# Author  :Yash Tripathi | Roll No: 24BCE10603
# What it does: Checks if Git is installed, shows its details,
#               and prints a philosophy note via a case statement.
# ==============================================================

# ANSI colours
GREEN='\033[0;32m'
BLUE='\033[0;34m'
YELLOW='\033[1;33m'
```

```
RED='\033[0;31m'
CYAN='\033[0;36m'
BOLD='\033[1m'
NC='\033[0m'

# The package we're inspecting — our chosen open-source software
PACKAGE="git"

echo -e "${BLUE}============================================================${NC}"
echo -e "${BOLD}   FOSS PACKAGE INSPECTOR — checking: $PACKAGE            ${NC}"
echo -e "${BLUE}============================================================${NC}"
echo ""

# if-then-else block: detect which package manager is present,
# then query it. &>/dev/null silences both stdout and stderr
# so failed checks don't print errors to the screen.

if command -v rpm &>/dev/null && rpm -q "$PACKAGE" &>/dev/null; then
   # RPM-based system (Fedora, RHEL, CentOS)
   echo -e "  ${GREEN}[FOUND]${NC} '$PACKAGE' is installed (RPM-based system)"
   echo ""
   echo -e "  ${CYAN}--- Package details from rpm -qi ---${NC}"
   rpm -qi "$PACKAGE" | grep -E 'Version|License|Summary|URL'

elif command -v dpkg &>/dev/null && dpkg -l "$PACKAGE" 2>/dev/null | grep -q "^ii"; then
   # Debian/Ubuntu-based system
   echo -e "  ${GREEN}[FOUND]${NC} '$PACKAGE' is installed (Debian-based system)"
   echo ""
   echo -e "  ${CYAN}--- Package details from dpkg -l ---${NC}"
   dpkg -l "$PACKAGE" | grep "^ii" | \
      awk '{print "  Version : "$3"\n  Package : "$2"\n  Desc    : "$5" "$6" "$7}'

elif command -v git &>/dev/null; then
   # Git present but not via a package manager (compiled from source)
   echo -e "  ${GREEN}[FOUND]${NC} '$PACKAGE' is installed (compiled from source)"
   echo ""
   echo -e "  ${CYAN}--- Version ---${NC}"
   git --version

else
   # Not found at all
   echo -e "  ${RED}[NOT FOUND]${NC} '$PACKAGE' is not installed on this system."
   echo ""
   echo -e "  To install Git:"
   echo -e "    ${YELLOW}Debian/Ubuntu :${NC} sudo apt install git"
   echo -e "    ${YELLOW}Fedora/RHEL   :${NC} sudo dnf install git"
   echo -e "    ${YELLOW}Arch Linux    :${NC} sudo pacman -S git"
fi

echo ""
echo -e "${BLUE}------------------------------------------------------------${NC}"
```

```
# case statement: prints an open-source philosophy note
# based on the package name. Each pattern matches a known FOSS tool.
echo -e "  ${GREEN}OPEN SOURCE PHILOSOPHY NOTE${NC}"
echo -e "${BLUE}------------------------------------------------------------${NC}"

case $PACKAGE in
   git)
      echo -e "  ${YELLOW}Git:${NC} Linus Torvalds built this in 2005 after a proprietary"
      echo -e "  tool was yanked from the Linux project. The whole point was"
      echo -e "  that no company could ever hold the kernel's history hostage"
      echo -e "  again. That design decision is why Git is distributed."
      ;;
   httpd|apache2)
      echo -e "  ${YELLOW}Apache:${NC} The web server that proved community-built software"
      echo -e "  could power the internet. At its peak it served over 60%"
      echo -e "  of all websites. Built openly, maintained collectively."
      ;;
   mysql|mariadb)
      echo -e "  ${YELLOW}MySQL:${NC} Its acquisition by Oracle sparked the MariaDB fork —"
      echo -e "  a reminder that even open-source projects can be threatened"
      echo -e "  by corporate ownership. The community voted with their feet."
      ;;
   python3|python)
      echo -e "  ${YELLOW}Python:${NC} Governed by a community foundation, not a company."
      echo -e "  The PSF licence is permissive enough that Python is now"
      echo -e "  the world's most used language. Openness drove adoption."
      ;;
   vlc)
      echo -e "  ${YELLOW}VLC:${NC} Started by students at a French university who wanted"
      echo -e "  to stream video over their campus network. No commercial"
      echo -e "  tool was open enough to adapt, so they built their own."
      ;;
   firefox)
      echo -e "  ${YELLOW}Firefox:${NC} Mozilla's answer to Internet Explorer's monopoly."
      echo -e "  A nonprofit building a browser to keep the web open."
      echo -e "  Without Firefox, the web would look very different today."
      ;;
   *)
      echo -e "  ${YELLOW}$PACKAGE:${NC} An open-source tool built on the belief that"
      echo -e "  shared knowledge moves faster than proprietary knowledge."
      ;;
esac

echo ""
echo -e "${BLUE}============================================================${NC}"
```

```
yash@Yash:/mnt/c/Users/yasht/Desktop/College/5th sem/Open source/project of linux vityarthi$ bash myscript2.sh
==============================================================
   FOSS PACKAGE INSPECTOR — checking: git
==============================================================

  [FOUND] 'git' is installed (Debian-based system)

  --- Package details from dpkg -l ---
  Version : 1:2.43.0-1ubuntu7.3
  Package : git
  Desc    : fast, scalable, distributed

--------------------------------------------------------------
  OPEN SOURCE PHILOSOPHY NOTE
--------------------------------------------------------------
  Git: Linus Torvalds built this in 2005 after a proprietary
  tool was yanked from the Linux project. The whole point was
  that no company could ever hold the kernel's history hostage
  again. That design decision is why Git is distributed.

==============================================================
yash@Yash:/mnt/c/Users/yasht/Desktop/College/5th sem/Open source/project of linux vityarthi$ |
```

Script 3 — Disk and Permission Auditor

Explanation:

This script loops through a predefined array of important system directories and reports the permissions, ownership, and disk usage of each. It then separately audits Git-specific configuration file locations on the system.

Concepts demonstrated

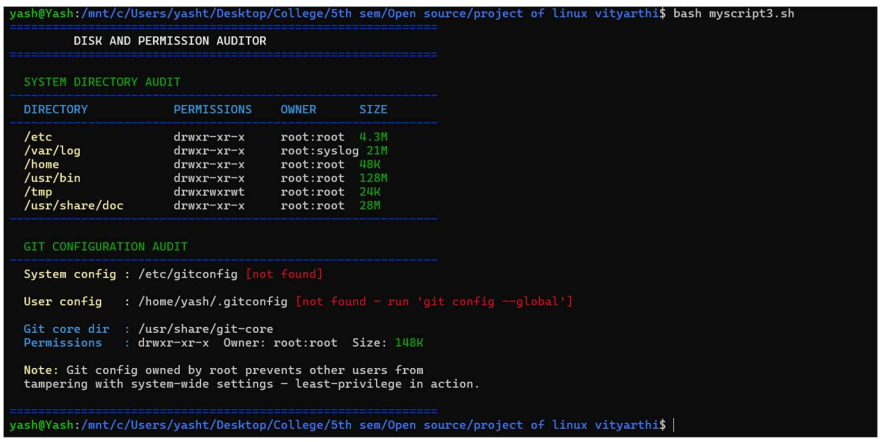
- for loop — iterates over the DIRS array using the ${DIRS[@]} expansion syntax.
- Array declaration — DIRS="("/etc" "/var/log" ...) holds multiple paths for the loop to process.
- ls -ld with awk — extracts the permissions string and owner:group from the long directory listing.
- du -sh with cut — produces a human-readable size figure by cutting the tab-separated du output.
- [-d] test — checks that each path is a real directory before trying to read it.

Code:

```
#!/bin/bash
# ==============================================================
# Script 3: Disk and Permission Auditor
# Course  : Open Source Software — VITyarthi
# Author  : Yash Tripathi | Roll No: 24BCE10603
# What it does: Loops through key system directories and reports
#               permissions, ownership, and size for each.
#               Also checks Git's own config file locations.
# ==============================================================

# ANSI colours
GREEN='\033[0;32m'
BLUE='\033[0;34m'
YELLOW='\033[1;33m'
RED='\033[0;31m'
CYAN='\033[0;36m'
BOLD='\033[1m'
NC='\033[0m'

# Array of important directories to audit
# These are standard locations on any Linux system
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/usr/share/doc")

echo -e "${BLUE}==============================================================${NC}"
echo -e "${BOLD}        DISK AND PERMISSION AUDITOR                    ${NC}"
echo -e "${BLUE}==============================================================${NC}"
echo ""
echo -e "  ${GREEN}SYSTEM DIRECTORY AUDIT${NC}"
```

```
echo -e "${BLUE}------------------------------------------------------------${NC}"
printf "  ${CYAN}%-20s %-14s %-10s %-8s${NC}\n" "DIRECTORY" "PERMISSIONS" "OWNER" "SIZE"
echo -e "${BLUE}------------------------------------------------------------${NC}"

# for loop — iterates over every entry in the DIRS array
# ${DIRS[@]} expands the full array; quotes handle paths with spaces
for DIR in "${DIRS[@]}"; do

   # [ -d "$DIR" ] tests whether the path exists and is a directory
   if [ -d "$DIR" ]; then
      # awk pulls out the permissions string ($1) and owner:group ($3:$4)
      PERMS=$(ls -ld "$DIR" | awk '{print $1}')
      OWNER=$(ls -ld "$DIR" | awk '{print $3":"$4}')
      # du -sh gives human-readable size; cut -f1 takes just the number
      SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)

      printf "  ${YELLOW}%-20s${NC} %-14s %-10s ${GREEN}%-8s${NC}\n" \
            "$DIR" "$PERMS" "$OWNER" "$SIZE"
   else
      # Directory doesn't exist on this particular system
      printf "  ${RED}%-20s${NC} %s\n" "$DIR" "[not found on this system]"
   fi
done

echo -e "${BLUE}------------------------------------------------------------${NC}"
echo ""

# Git-specific config directory audit
# This section checks the three tiers of Git configuration on Linux
echo -e "  ${GREEN}GIT CONFIGURATION AUDIT${NC}"
echo -e "${BLUE}------------------------------------------------------------${NC}"

GIT_SYSTEM="/etc/gitconfig"
GIT_USER="$HOME/.gitconfig"
GIT_CORE="/usr/share/git-core"

# Check system-wide git config
if [ -f "$GIT_SYSTEM" ]; then
   PERMS=$(ls -l "$GIT_SYSTEM" | awk '{print $1}')
   OWNER=$(ls -l "$GIT_SYSTEM" | awk '{print $3":"$4}')
   echo -e "  ${CYAN}System config :${NC} $GIT_SYSTEM"
   echo -e "  ${CYAN}Permissions   :${NC} $PERMS  Owner: $OWNER"
else
   echo -e "  ${YELLOW}System config :${NC} $GIT_SYSTEM ${RED}[not found]${NC}"
fi

echo ""

# Check user-level git config
if [ -f "$GIT_USER" ]; then
   PERMS=$(ls -l "$GIT_USER" | awk '{print $1}')
   OWNER=$(ls -l "$GIT_USER" | awk '{print $3":"$4}')
```

```
    echo -e "  ${CYAN}User config   :${NC} $GIT_USER"
    echo -e "  ${CYAN}Permissions   :${NC} $PERMS  Owner: $OWNER"
else
    echo -e "  ${YELLOW}User config   :${NC} $GIT_USER ${RED}[not found — run 'git config --global']${NC}"
fi

echo ""

# Check Git's shared template/hooks directory
if [ -d "$GIT_CORE" ]; then
    SIZE=$(du -sh "$GIT_CORE" 2>/dev/null | cut -f1)
    PERMS=$(ls -ld "$GIT_CORE" | awk '{print $1}')
    OWNER=$(ls -ld "$GIT_CORE" | awk '{print $3":"$4}')
    echo -e "  ${CYAN}Git core dir  :${NC} $GIT_CORE"
    echo -e "  ${CYAN}Permissions   :${NC} $PERMS  Owner: $OWNER  Size: ${GREEN}$SIZE${NC}"
    echo ""
    echo -e "  ${YELLOW}Note:${NC} Git config owned by root prevents other users from"
    echo -e "  tampering with system-wide settings — least-privilege in action."
else
    echo -e "  ${YELLOW}Git core dir  :${NC} $GIT_CORE ${RED}[not found]${NC}"
fi

echo ""
echo -e "${BLUE}============================================================${NC}"
```



Script 4 — Log File Analyzer

Explanation:

This script takes a log file path and an optional keyword as command-line arguments, scans the file line by line counting matches, implements a retry loop for empty files, and prints a summary with the last five matching lines.

Concepts demonstrated

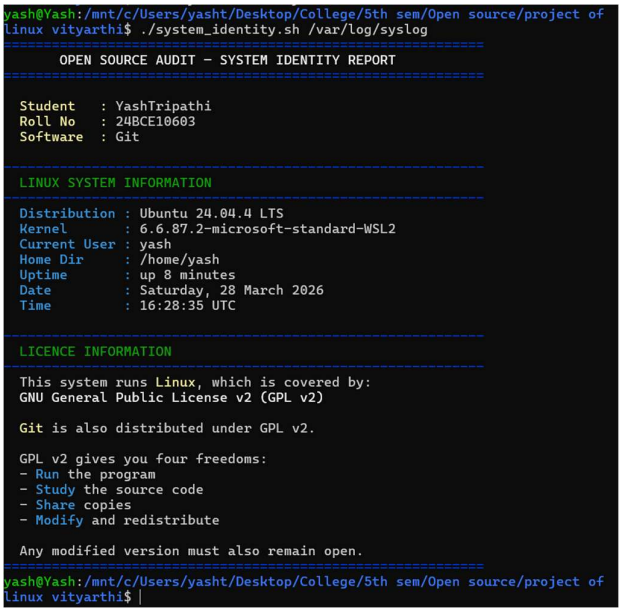
- Command-line arguments — $1 and $2 take the log file and keyword; ${2:-"error"} provides a default value if none is given.
- while IFS= read -r — reads the log file line by line, with IFS= and -r preserving whitespace and backslashes correctly.
- Counter variable — COUNT=$(($COUNT + 1)) uses arithmetic expansion to track keyword occurrences.
- Do-while simulation — a while true loop with break implements retry behaviour when the file appears empty.
- grep -iq with tail — extracts and displays the last five matching lines for context.

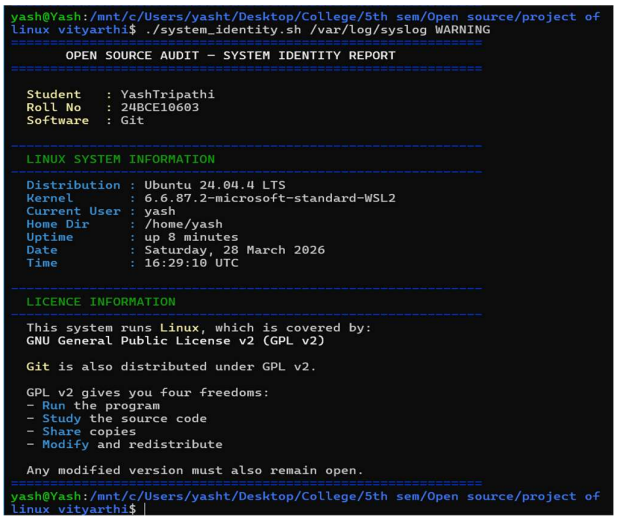
Code:

```
#!/bin/bash
# ============================================================
# Script 3: Disk and Permission Auditor
# Course  : Open Source Software — VITyarthi
# Author  : Yash Tripathi | Roll No: 24BCE10603
# What it does: Loops through key system directories and reports
#               permissions, ownership, and size for each.
#               Also checks Git's own config file locations.
# ============================================================

# ANSI colours
GREEN='\033[0;32m'
BLUE='\033[0;34m'
YELLOW='\033[1;33m'
RED='\033[0;31m'
CYAN='\033[0;36m'
BOLD='\033[1m'
NC='\033[0m'

# Command-line arguments
# $1 is the log file path; $2 is the keyword (default: "error")
LOGFILE=$1
KEYWORD=${2:-"error"}

# Counter and retry variables
COUNT=0
RETRY_LIMIT=3
RETRY_COUNT=0

echo -e "${BLUE}============================================================${NC}"
echo -e "${BOLD}        LOG FILE ANALYZER                          ${NC}"
echo -e "${BLUE}============================================================${NC}"
echo ""

# Validate that the user actually gave us a file path
if [ -z "$LOGFILE" ]; then
   echo -e "  ${RED}ERROR:${NC} No log file specified."
   echo -e "  Usage  : $0 /path/to/logfile [keyword]"
   echo -e "  Example: $0 /var/log/syslog error"
   exit 1
fi

# Check the file exists — [ -f ] tests for a regular file
if [ ! -f "$LOGFILE" ]; then
   echo -e "  ${RED}ERROR:${NC} '$LOGFILE' not found."
   echo -e "  Check the path and try again."
   exit 1
fi

echo -e "  ${CYAN}Log File :${NC} $LOGFILE"
echo -e "  ${CYAN}Keyword  :${NC} '${YELLOW}$KEYWORD${NC}'"
```

```
echo ""
echo -e "${BLUE}------------------------------------------------------------${NC}"

# Do-while style retry loop — Bash has no native do-while,
# so we use "while true" and break out once the file has content.
# This handles the edge case where a log file exists but is empty.
while true; do
    if [ -s "$LOGFILE" ]; then
        break   # File is non-empty — proceed
    fi

    RETRY_COUNT=$((RETRY_COUNT + 1))

    if [ $RETRY_COUNT -ge $RETRY_LIMIT ]; then
        echo -e "  ${YELLOW}WARNING:${NC} File appears empty after $RETRY_LIMIT checks."
        echo -e "  Continuing — result will show 0 matches."
        break
    fi

    echo -e "  File looks empty. Retrying ($RETRY_COUNT/$RETRY_LIMIT)..."
    sleep 1
done

echo -e "  Scanning for keyword '${YELLOW}$KEYWORD${NC}'..."
echo ""

# while IFS= read -r reads the log file one line at a time.
# IFS= prevents leading/trailing whitespace from being stripped.
# -r prevents backslashes from being interpreted as escape characters.
while IFS= read -r LINE; do
    # grep -iq does a case-insensitive quiet match on each line
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))   # arithmetic expansion increments the counter
    fi
done < "$LOGFILE"

# Print the summary
echo -e "${BLUE}------------------------------------------------------------${NC}"
echo -e "  ${GREEN}SCAN SUMMARY${NC}"
echo -e "${BLUE}------------------------------------------------------------${NC}"

if [ $COUNT -gt 0 ]; then
    echo -e "  Keyword '${YELLOW}$KEYWORD${NC}' found ${GREEN}${BOLD}$COUNT${NC} time(s) in:"
    echo -e "  $LOGFILE"
    echo ""
    echo -e "  ${CYAN}Last 5 matching lines:${NC}"
    echo -e "${BLUE}------------------------------------------------------------${NC}"
    # grep + tail extracts the last 5 lines containing the keyword
    grep -i "$KEYWORD" "$LOGFILE" | tail -5 | while IFS= read -r MATCH; do
        echo -e "  ${YELLOW}>>>${NC} $MATCH"
    done
else
```

```
    echo -e "  Keyword '${YELLOW}$KEYWORD${NC}' was ${RED}not found${NC} in $LOGFILE."
    echo -e "  Try a different keyword: ${CYAN}warning, fail, denied, critical${NC}"
fi

echo ""
echo -e "${BLUE}============================================================${NC}"
```





Script 5 — Open Source Manifesto Generator

Explaination:

This script asks the user three interactive questions about their open-source philosophy, collects system metadata, generates a formatted manifesto file using a heredoc, and saves it with a timestamped filename.

Concepts demonstrated

- **read command** — read -p prompts the user and stores input in variables like TOOL, FREEDOM, and BUILD.
- **Alias as function** — today() function mimics the behaviour of an alias, showing how to wrap simple commands for reuse.
- **Heredoc (<<EOF)** — writes multi-line text to a file cleanly without multiple echo statements; variables inside are expanded automatically.

- **Command substitution** — $(whoami), $(hostname), $(date +%Y%m%d) capture system information at runtime.
- **Filename generation** — manifesto_${USER_NAME}_$(date +%Y%m%d).txt creates a unique, traceable output file per user per day.
- **cat with redirection** — cat > "$OUTPUT" << EOF combines file creation and heredoc content writing.
- **ANSI colour codes** — \033[0;32m etc. add coloured terminal output for better user experience.

Code:

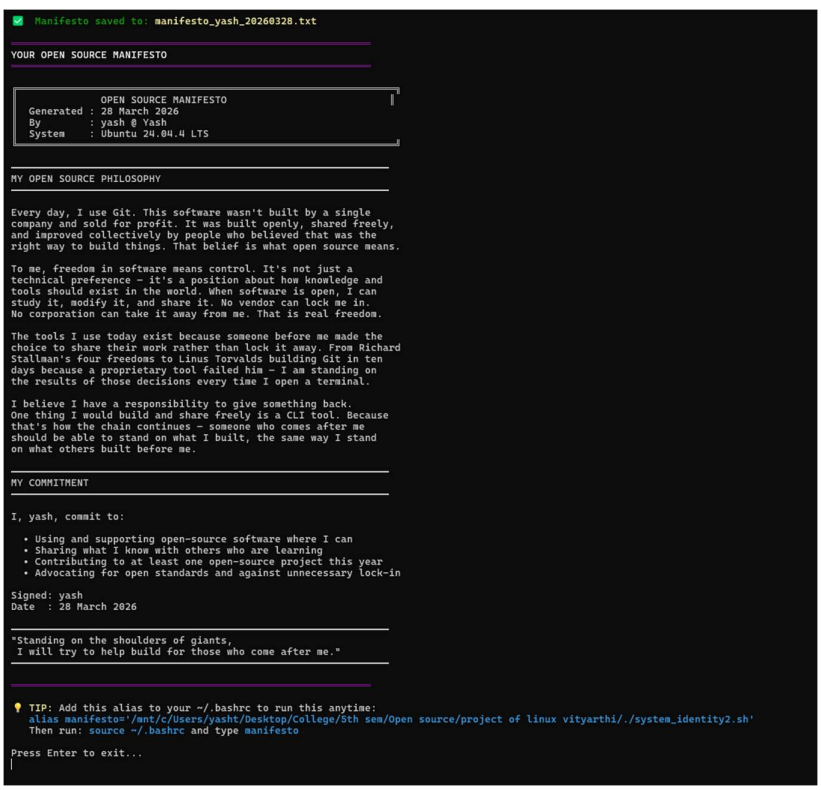
```
#!/bin/bash
# ============================================================
# Script 3: Disk and Permission Auditor
# Course  : Open Source Software — VITyarthi
# Author  : Yash Tripathi | Roll No: 24BCE10603
# What it does: Loops through key system directories and reports
#               permissions, ownership, and size for each.
#               Also checks Git's own config file locations.
# ============================================================


# ANSI colours
GREEN='\033[0;32m'
BLUE='\033[0;34m'
YELLOW='\033[1;33m'
PURPLE='\033[0;35m'
CYAN='\033[0;36m'
BOLD='\033[1m'
NC='\033[0m'

# Alias concept demonstrated as a named function.
# In an interactive shell you'd write: alias today='date "+%d %B %Y"'
# Here we wrap the same logic in a function so the script can call it.
today() {
   date '+%d %B %Y'
}

clear
echo -e
"${BLUE}═══════════════════════════════════════════════════════════════════════════════
═${NC}"
echo -e "${BOLD}        OPEN SOURCE MANIFESTO GENERATOR                  ${NC}"
echo -e
"${BLUE}═══════════════════════════════════════════════════════════════════════════════
═${NC}"
echo ""
echo -e "${YELLOW}This tool asks you three questions about your relationship${NC}"
echo -e "${YELLOW}with open-source software, then generates a personal      ${NC}"
echo -e "${YELLOW}philosophy statement and saves it to a file.              ${NC}"
echo ""
echo -e "Press ${CYAN}Enter${NC} to begin..."
read

clear

# ── Collect user input ──────────────────────────────────────────────

echo -e
"${BLUE}═══════════════════════════════════════════════════════════════════════════════
═${NC}"
echo -e "${GREEN}Question 1 of 3${NC}"
```

```
echo -e
"${BLUE}═══════════════════════════════════════════════════════════════════════════
═${NC}"
echo ""
echo -e "${YELLOW}Name one open-source tool you use every day:${NC}"
echo -e "(Examples: Git, Firefox, Linux, VLC, Python, LibreOffice)"
echo ""
# read -p prompts inline and stores the response in the variable
read -p "➜ " TOOL
echo ""

echo -e
"${BLUE}═══════════════════════════════════════════════════════════════════════════
═${NC}"
echo -e "${GREEN}Question 2 of 3${NC}"
echo -e
"${BLUE}═══════════════════════════════════════════════════════════════════════════
═${NC}"
echo ""
echo -e "${YELLOW}In one word, what does 'freedom' mean to you in software?${NC}"
echo -e "(Examples: choice, transparency, control, privacy, community)"
echo ""
read -p "➜ " FREEDOM
echo ""

echo -e
"${BLUE}═══════════════════════════════════════════════════════════════════════════
═${NC}"
echo -e "${GREEN}Question 3 of 3${NC}"
echo -e
"${BLUE}═══════════════════════════════════════════════════════════════════════════
═${NC}"
echo ""
echo -e "${YELLOW}Name one thing you would build and share freely:${NC}"
echo -e "(Examples: a learning platform, a document editor, a CLI tool)"
echo ""
read -p "➜ " BUILD
echo ""

# ── Gather metadata ──────────────────────────────────────────────────────

DATE=$(today)          # uses our alias-style function
USER_NAME=$(whoami)
HOSTNAME=$(hostname)
DISTRO=$(grep -w "PRETTY_NAME" /etc/os-release 2>/dev/null \
        | cut -d= -f2 | tr -d "" \
        || echo "Linux")

# Output filename includes the username and today's date
OUTPUT="manifesto_${USER_NAME}_$(date +%Y%m%d).txt"

# ── Write manifesto using heredoc (<<EOF) ────────────────────────────────
# The heredoc writes everything between EOF markers to the file
# in one clean block — much neater than a chain of echo >> lines.
# Variables inside heredoc are still expanded automatically.

cat > "$OUTPUT" << EOF
╔═══════════════════════════════════════════════════════════════════════════╗
          OPEN SOURCE MANIFESTO                          ║
  Generated : $DATE
```

```
By       : $USER_NAME @ $HOSTNAME
System   : $DISTRO
```

```
────────────────────────────────────────────────────────────────────────────────
────────────────────────────────
MY OPEN SOURCE PHILOSOPHY
────────────────────────────────────────────────────────────────────────────────
────────────────────────────────

Every day, I use $TOOL. This software wasn't built by a single
company and sold for profit. It was built openly, shared freely,
and improved collectively by people who believed that was the
right way to build things. That belief is what open source means.

To me, freedom in software means $FREEDOM. It's not just a
technical preference — it's a position about how knowledge and
tools should exist in the world. When software is open, I can
study it, modify it, and share it. No vendor can lock me in.
No corporation can take it away from me. That is real freedom.

The tools I use today exist because someone before me made the
choice to share their work rather than lock it away. From Richard
Stallman's four freedoms to Linus Torvalds building Git in ten
days because a proprietary tool failed him — I am standing on
the results of those decisions every time I open a terminal.

I believe I have a responsibility to give something back.
One thing I would build and share freely is $BUILD. Because
that's how the chain continues — someone who comes after me
should be able to stand on what I built, the same way I stand
on what others built before me.

────────────────────────────────────────────────────────────────────────────────
────────────────────────────────
MY COMMITMENT
────────────────────────────────────────────────────────────────────────────────
────────────────────────────────

I, $USER_NAME, commit to:

  • Using and supporting open-source software where I can
  • Sharing what I know with others who are learning
  • Contributing to at least one open-source project this year
  • Advocating for open standards and against unnecessary lock-in

Signed: $USER_NAME
Date  : $DATE

────────────────────────────────────────────────────────────────────────────────
────────────────────────────────
"Standing on the shoulders of giants,
 I will try to help build for those who come after me."
────────────────────────────────────────────────────────────────────────────────
────────────────────────────────
EOF

# ── Display the result ──────────────────────────────────────────────────────
```

```
clear
echo -e "${GREEN}✅ Manifesto saved to: ${YELLOW}$OUTPUT${NC}"
echo ""
echo -e
"${PURPLE}═══════════════════════════════════════════════════════════════════
═══${NC}"
echo -e "${BOLD}YOUR OPEN SOURCE MANIFESTO${NC}"
echo -e
"${PURPLE}═══════════════════════════════════════════════════════════════════
═══${NC}"
echo ""

# cat displays the saved file back to the screen
cat "$OUTPUT"

echo ""
echo -e
"${PURPLE}═══════════════════════════════════════════════════════════════════
═══${NC}"
echo ""

# Alias tip — shows the user how to make this a quick command
echo -e "${YELLOW}💡 TIP:${NC} Add this alias to your ~/.bashrc to run this anytime:"
echo -e "   ${CYAN}alias manifesto='$PWD/$0'${NC}"
echo -e "   Then run: ${CYAN}source ~/.bashrc${NC} and type ${CYAN}manifesto${NC}"
echo ""
echo -e "Press Enter to exit..."
read
```



Conclusion

This audit of Git goes beyond just understanding it as a version control tool — it reveals how deeply its design, licence, and philosophy are connected to the broader idea of open source. Git was not created in isolation; it was a direct response to a real-world problem involving dependency on proprietary software. That origin is reflected in its distributed nature, which removes central control and ensures that no single entity can restrict access or ownership again.

From a technical perspective, Git demonstrates how efficient design — minimal dependencies, strong integration with Linux systems, and powerful branching capabilities — can scale from individual developers to massive global projects. Its presence across modern development workflows, from local repositories to CI/CD pipelines, shows that it is not just a tool but a foundational layer of today's software ecosystem.

The GPL v2 licence plays a critical role in preserving this openness. By enforcing copyleft, it ensures that the freedoms associated with Git are not lost over time. This creates a system where innovation can continue without the risk of being closed off, aligning with the core values of free software.

At the same time, this audit highlights that open source is not just a technical model but also an ethical one. Questions around contribution, fairness, and community responsibility remain important, especially as large organisations increasingly rely on open-source tools. Git stands as a strong example of how community-driven development, when supported by the right licence and governance model, can produce software that is both highly practical and philosophically meaningful.

In conclusion, Git represents more than just a successful software project — it represents a shift in how software is built, shared, and controlled. Its impact on modern development is undeniable, and its principles continue to shape the future of open-source innovation.

References

The following were used for research. Everything in this report is written in my own words.

- GNU Project. The Free Software Definition. gnu.org/philosophy/free-sw.html
- Open Source Initiative. The Open Source Definition. opensource.org/osd
- Raymond, E. S. The Cathedral and the Bazaar. catb.org/~esr/writings/cathedral-bazaar
- Torvalds, L. Just for Fun: The Story of an Accidental Revolutionary. HarperBusiness, 2001.
- Chacon, S. & Straub, B. Pro Git (2nd ed.). Apress, 2014. git-scm.com/book
- Shotts, W. The Linux Command Line. linuxcommand.org
- GNU Bash Manual. gnu.org/software/bash/manual
- SPDX License List — GPL-2.0-only. spdx.org/licenses/GPL-2.0-only.html
- Git official documentation. git-scm.com/docs
- Linux kernel mailing list archives. lore.kernel.org
- Linux man pages. man7.org/linux/man-pages